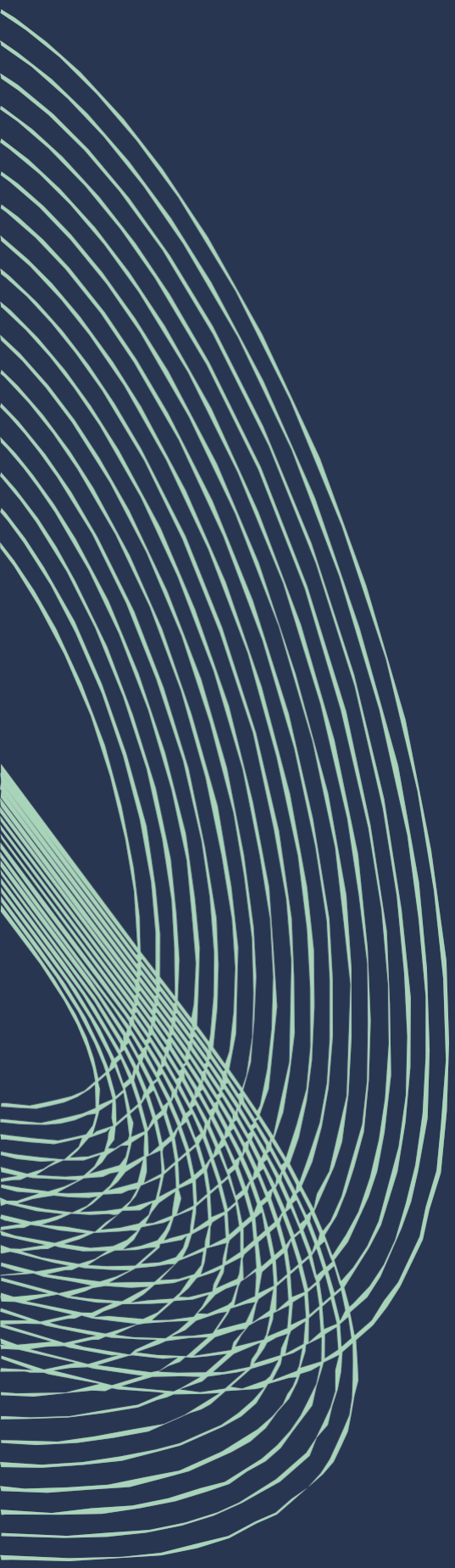
ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR
Filière : Années Préparatoires 1ère Année — G11
Module : Langage de Programmation C



Système de gestion de location de voitures

Rapport de Mini-Projet

Réalisé par: Sabrine MEDKOURI-ZIDI
Encadré par: M. Anas MOUFLIH
Durée du projet: 1 Semaine
Année académique: 2025/2026

SOMMAIRE

Introduction	3
Objectifs du projet	4
Analyse des données	5
Règles de calcul	6
Conception modulaire	7
Fonctionnalités implémentées	8
Implémentation	9
Exemple d’exécution	15
Conclusion globale	17

1. INTRODUCTION

Dans un contexte où la numérisation des services devient incontournable, la gestion manuelle des opérations de location de voitures engendre des inefficacités et des risques d'erreurs. Ce projet s'inscrit dans ce cadre en proposant une solution informatisée développée en langage C.

L'application vise à automatiser l'ensemble du cycle de gestion d'une agence de location : enregistrement des locations, calcul automatique des coûts, tri, recherche et génération de rapports de rentabilité.

Ce rapport présente la démarche adoptée, allant de l'analyse des besoins jusqu'à la réalisation complète du programme, en passant par la conception des structures de données et des algorithmes.

2. Objectifs du projet

Ce projet a pour objectif de concevoir et développer une application console interactive en langage C permettant de gérer les opérations de location de véhicules d'une agence. Les fonctionnalités attendues sont les suivantes :

- **Enregistrer** une location
- **Calculer automatiquement** le tarif journalier et l'assurance selon la catégorie du véhicule
- **Déterminer** le coût total de la location en fonction de la durée
- **Afficher un rapport de rentabilité** trié par coût total décroissant
- **Afficher la liste** des locations triée alphabétiquement par nom de client
- **Rechercher** une location par nom de client
- **Modifier** la durée d'une location (par immatriculation) et recalculer le coût total

Contrainte technique : Le tri et toute manipulation de tableaux doivent utiliser des pointeurs. Le code doit être organisé en fonctions modulaires. La taille maximale du tableau est fixée par #define MAX 30.

3. Analyse des données

Le système repose sur une structure unique nommée Location qui centralise toutes les informations nécessaires au traitement d'une opération de location.

Structure Location — champs et types :

Champ	Type C	Description
immatriculation	char[20]	Numéro de plaque du véhicule
marque	char[20]	Marque du véhicule (ex : BMW, Dacia...)
categorie	char[20]	Catégorie : Economique, Standard ou Luxe
client	char[50]	Nom du client (utilisé pour tri et recherche)
jours	int	Durée de la location en jours
tarif	float	Tarif journalier en MAD (calculé auto)
assurance	float	Taux d'assurance (calculé auto selon catégorie)
total	float	Coût total de la location en MAD

L'utilisation d'une structure unique simplifie le passage des données entre fonctions via des pointeurs, conformément aux exigences du projet.

4. Règle de calcul

Les tarifs et taux d'assurance sont déterminés automatiquement selon la catégorie du véhicule, conformément au cahier des charges :

Catégorie	Tarif journalier	Taux assurance	Exemple (5 jours)
Économique	200 MAD/j	5 %	1 050 MAD
Standard	350 MAD/j	8 %	1 890 MAD
Luxe	600 MAD/j	12 %	3 360 MAD

Formule appliquée :

```
Coût total = (Jours × Tarif/j) + (Jours × Tarif/j × Taux d'assurance)
```

Cette formule est implémentée dans la fonction calculerTotal() qui reçoit un pointeur vers la structure Location et retourne le coût total sous forme de float.

Vérification avec l'exemple du rapport de rentabilité du sujet :

Client	Véhicule	Catégorie	Jours	Tarif	Total
Alami Sara	BMW 320	Luxe	5j	600 MAD	3 360 MAD
Benali Omar	Dacia Logan	Économique	7j	200 MAD	1 470 MAD
Chraibi Lina	Clio IV	Standard	3j	350 MAD	1 134 MAD

Chiffre d'affaires total : 5 964 MAD | Véhicules loués : 3

5. Conception Modulaire

Conformément aux exigences du sujet, le programme est structuré en fonctions indépendantes. Chaque fonction a un rôle précis et reçoit les données par pointeur.

Fonction	Rôle	Passage de paramètres
definirTarif()	Affecte tarif et taux selon catégorie	Pointeur Location *l
calculerTotal()	Calcule et retourne le coût total	Pointeur Location *l
enregistrer()	Saisie et enregistrement d'une location	Tableau + compteur *n
trierParTotal()	Tri décroissant par coût (pour rapport)	Tableau + taille n
trierParNom()	Tri alphabétique des clients (option 3)	Tableau + taille n
rapport()	Affiche le rapport de rentabilité (option 2)	Tableau + taille n
listeAlphabetique()	Affiche la liste triée par client (option 3)	Tableau + taille n
rechercher()	Recherche par nom de client (option 4)	Tableau + taille n
modifier()	Modifie la durée par immatriculation (option 5)	Tableau + taille n
main()	Menu principal et boucle de contrôle	—

Le menu principal utilise une boucle do...while avec un switch permettant l'appel de chaque fonction selon le choix de l'utilisateur (options 1 à 5, 0 pour quitter).

6. Fonctionnalités

Le programme permet de gérer un ensemble de voitures de location à travers un menu interactif :

- **Choix 1: Enregistrement d’une location:**
  - Saisie des informations : immatriculation, marque, catégorie, client, jours
  - Calcul automatique du coût total

- **Choix 2: Affichage du rapport**
  - Affichage trié par coût total décroissant
  - Affiche : client, véhicule, catégorie, jours, tarif, assurance, total
  - Calcul du chiffre d’affaires total

- **Choix 3: Affichage du liste**
  - Tri alphabetique des locations selon le nom du client

- **Choix 4: Recherche d’une location par client**
  - Recherche d’une location par nom du client

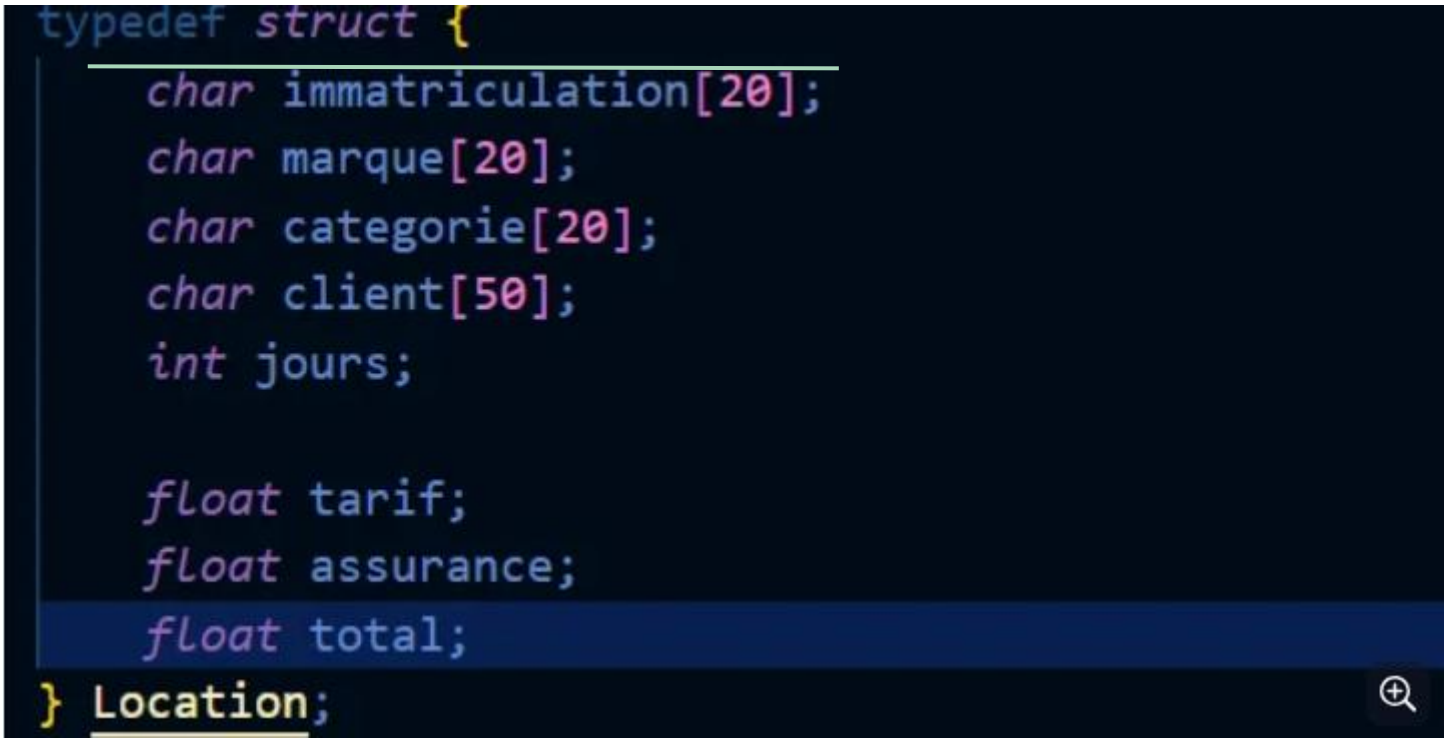
*

- **Choix 5: Modification**
  - Modifier la durée par immatriculation
  - Recalcul automatique du coût

- **Choix 6: Quitter**
  - Sortie du programme

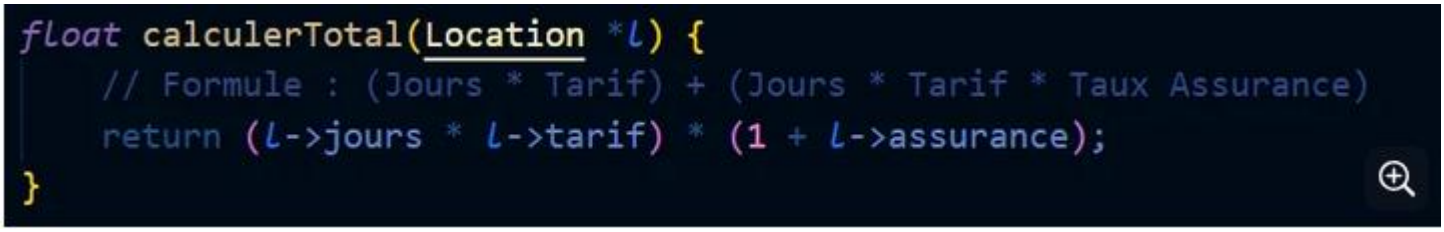
7. Implémentation (Explication du code)

◆ Structure principale



- cette partie correspond a la definition d'une structure qui s'apelle location grace au typedef.

◆ Fonction calculerTotal



- ici on a Utilise **une fonction** nommer **calculerTotal** avec **un pointeur** (l->) pour calculer **le prix total.**

◆ Fonction definirTarif



```
void definirTarif(Location *l) {
    if (strcmp(l->categorie, "Economique") == 0) {
        l->tarif = 200;
        l->assurance = 0.05;
    } else if (strcmp(l->categorie, "Standard") == 0) {
        l->tarif = 350;
        l->assurance = 0.08;
    } else if (strcmp(l->categorie, "Luxe") == 0) {
        l->tarif = 600;
        l->assurance = 0.12;
    }
}
```

- ici on a Utilise une fonction nommer definirTarif avec un pointeur (l->), avec des “if” et “else if” pour designer chaque class et son prix avec son taux d’assurance.
- (strcmp est pour comparer le contenu des chaînes, par exemple dans la deuxième ligne on a **(strcmp(l - >categorie, “Economique”) == 0), ça veut dire que le pointeur qui pointe vers ce qui a été enregistré dans "categorie" va être comparé avec "Economique".**)

◆ Fonction enregistrer

```c
    Location *l = &tab[*n];

    printf("Immatriculation: ");
    scanf("%s", l->immatriculation);

    printf("Marque (Dacia, BMW, Clio...): ");
    scanf(" %[^\n]", l->marque);

    printf("Categorie (Economique / Standard / Luxe): ");
    scanf("%s", l->categorie);

    printf("Nom du client: ");
    scanf(" %[^\n]", l->client);

    printf("Nombre de jours: ");
    scanf("%d", &l->jours);

    definirTarif(l);
    l->total = calculerTotal(l);

    printf(">>> Prix total de la location: %.2f MAD\n", l->total);
    (*n)++;
}
```

Ici, on a un **pointeur** qui **pointe** vers la **nouvelle case**. Ensuite, on accède aux données en lecture avec scanf, puis on **calcule** le tarif avec la formule (où l-> permet d'accéder au total par le pointeur).
Concernant **%[^\n]**, il sert à lire **toute la saisie** jusqu'au **retour** à la ligne, c'est-à-dire une chaîne de caractères **incluant** les **espaces**.

◆ Fonction trierParTotal

```c
void trierParTotal(Location tab[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (tab[i].total < tab[j].total) {
                Location temp = tab[i];
                tab[i] = tab[j];
                tab[j] = temp;
            }
        }
    }
}
```

ci, on a une nouvelle **fonction** nommée **trierParTotal** où on utilise **la structure Location**.
Pour **la première** boucle **for**, on utilise **i < n - 1** car il n'y a pas d'autres éléments après le dernier pour comparer, donc on doit s'arrêter **avant le dernier élément**.
Pour **la deuxième** boucle **for**, elle sert à **comparer** l'élément avec ceux qui viennent **après**, et pour éviter de le comparer avec **lui-même** ou de refaire **la même comparaison,** on commence avec **j = i + 1.**

◆ Fonction rapport

```
void rapport(Location tab[], int n) {
    trierParTotal(tab, n);
    float chiffreTotal = 0;
    printf("\n=========================================================================\n");
    printf("--- RAPPORT DE RENTABILITE ---\n");
    printf("=========================================================================\n");
    printf("Rang | Client | Vehicule | Cat | Jours | Tarif | Assurance | Total\n");
    printf("-------------------------------------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        float tarifCumule = tab[i].tarif * tab[i].jours;
        float montantAssurance = tarifCumule * tab[i].assurance;
        printf("%d | %s | %s | %s | %dj | %.0f | %.0f | %.2f MAD\n", i + 1, tab[i].client, tab[i].marque, tab[i].categorie,
               tab[i].jours, tarifCumule, montantAssurance, tab[i].total);
        chiffreTotal += tab[i].total;
    }
    printf("-------------------------------------------------------------------------\n");
    printf("Chiffre d'affaires total : %.2f MAD | Vehicules loues: %d\n", chiffreTotal, n);
    printf("=========================================================================\n");
}
```

Ici, on a une **fonction** nommée **rapport** qui permet **d'afficher** le rapport avec les tarifs réels et l'assurance. La boucle **for** est utilisée pour calculer **le prix total** : (tarif * jours) * assurance.

◆ Fonction trierParNom

```
void trierParNom(Location tab[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (strcmp(tab[i].client, tab[j].client) > 0) {
                Location temp = tab[i];
                tab[i] = tab[j];
                tab[j] = temp;
            }
        }
    }
}
```

Dans cette partie, on a une autre **fonction** nommée **trierParNom** qui permet de **trier** les données **selon le nom du client** (par ordre alphabétique). Les boucles **for** ont le même rôle que précédemment **(la comparaison)**.

◆ Fonction listeAlphabetique

client - voiture - total

```
void listeAlphabetique(Location tab[], int n) {
    trierParNom(tab, n);
    printf("\n--- LISTE PAR CLIENT ---\n");
    for (int i = 0; i < n; i++) {
        printf("%s - %s - %.2f MAD\n", tab[i].client, tab[i].marque, tab[i].total);
    }
}
```

Ici, on a une **fonction** appelée **ListeAlphabetique**. On appelle la **fonction précédente trierParNom** afin d'afficher la liste des clients par **ordre alphabétique**.

◆ Fonction rechercher

```
void rechercher(Location tab[], int n) {
    char nom[50];
    printf("Nom du client: ");
    scanf(" %[^\n]", nom);
    for (int i = 0; i < n; i++) {
        if (strcmp(tab[i].client, nom) == 0) {
            printf("Trouve: %s - %s - %.2f MAD\n", tab[i].client, tab[i].marque, tab[i].total);
            return;
        }
    }
    printf("Client non trouve.\n");
}
```

Dans cette partie, on a une fonction nommée **recherche** qui sert à **rechercher les noms**, avec une condition **d'invalidation** du **nom**.

◆ Fonction modifier

```
void modifier(Location tab[], int n) {
    char imm[20];
    printf("Immatriculation: ");
    scanf("%s", imm);
    for (int i = 0; i < n; i++) {
        if (strcmp(tab[i].immatriculation, imm) == 0) {
            printf("Nouvelle duree (jours): ");
            scanf("%d", &tab[i].jours);
            tab[i].total = calculerTotal(&tab[i]);
            printf("Nouveau total mis a jour: %.2f MAD\n", tab[i].total);
            return;
        }
    }
    printf("Location non trouvee.\n");
}
```

Ici, on effectue une **recherche** par **immatriculation**, suivie d'une **modification** du **nombre de jours**, puis d'un **recalcul** à l'aide d'une **fonction** appelée **modifier**. Une boucle **for**, **contenant** une condition **if**, permet de **comparer** les **immatriculations** grâce à **strcmp** afin d'effectuer la **modification** de la durée.

◆ main()

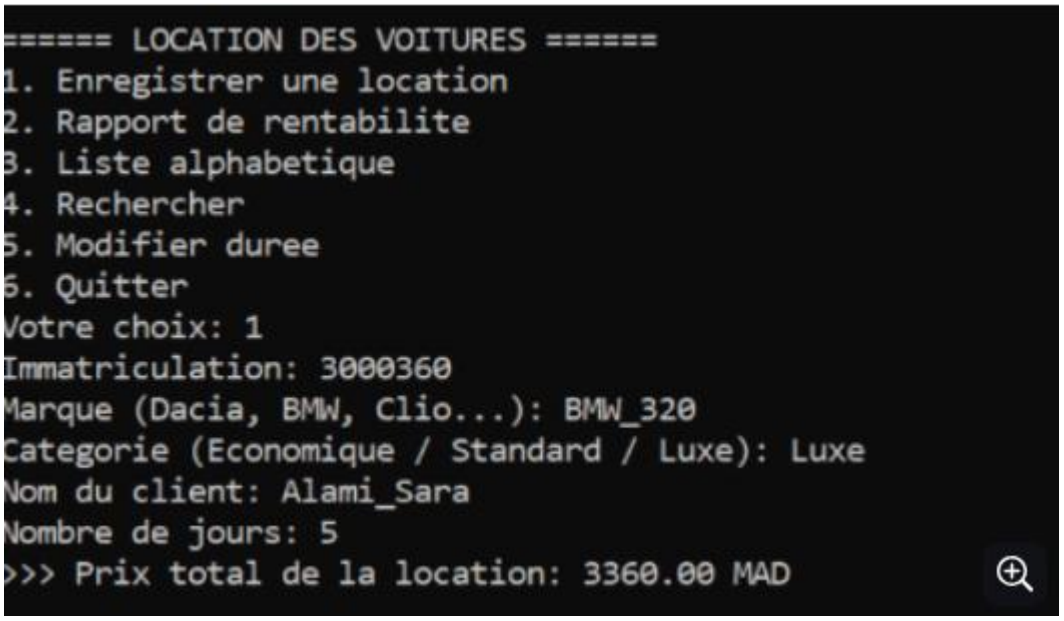
```c
int main() {
    Location tab[MAX];
    int n = 0, choix;
    do {
        printf("\n====== LOCATION DES VOITURES ======\n");
        printf("1. Enregistrer une location\n2. Rapport de rentabilite\n3. Liste alphabetique\n4. Rechercher\n5. Modifier duree\n6. Quitter\n");
        printf("Votre choix: ");
        scanf("%d", &choix);
        switch (choix) {
            case 1: enregistrer(tab, &n); break;
            case 2: if(n>0) rapport(tab, n); else printf("Aucune donnée.\n"); break;
            case 3: listeAlphabetique(tab, n); break;
            case 4: rechercher(tab, n); break;
            case 5: modifier(tab, n); break;
            case 6: printf("Au revoir !\n"); break;

            default: printf("Choix invalide.\n");
        }
    } while (choix != 6);
    return 0;
}
```

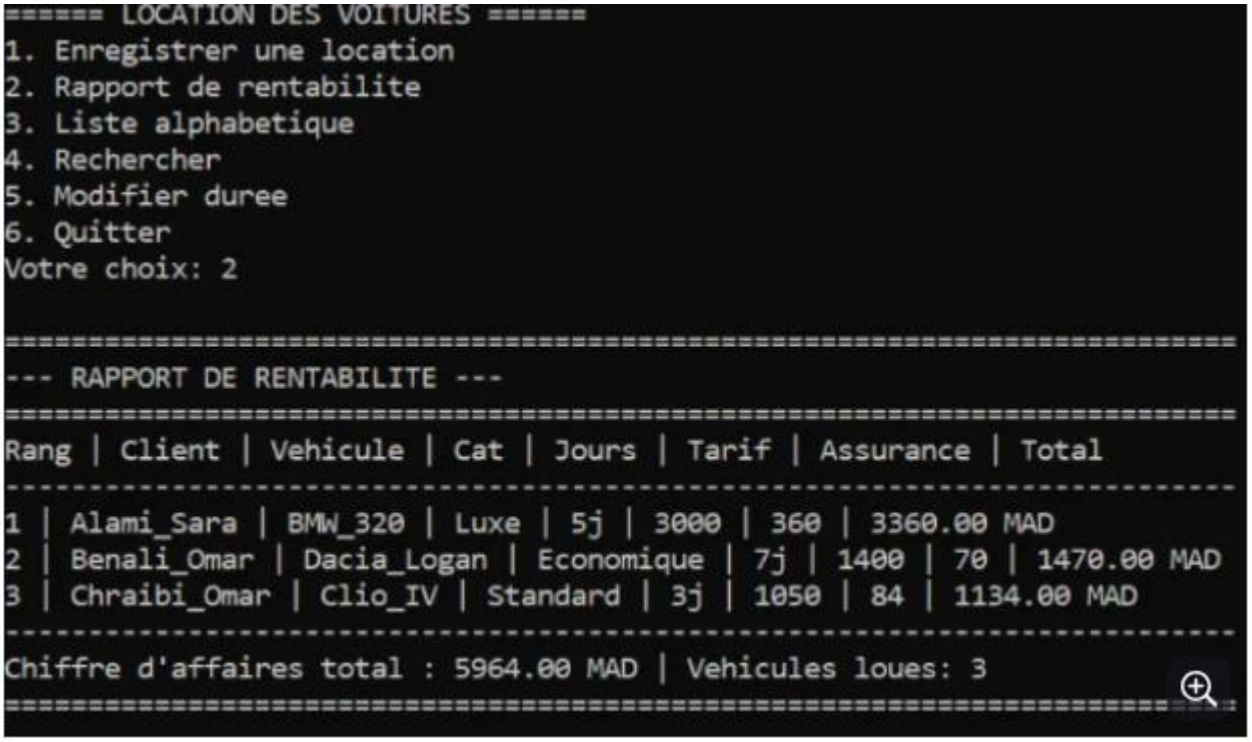
Ici, on a **activé** un menu interactif en utilisant **do...while**, avec un **switch** pour gérer **chaque fonction** selon le **choix** de l'utilisateur.

8. Exemple d’exécution:

Choix1:



Choix2:



Choix3:

```
====== LOCATION DES VOITURES ======
1. Enregistrer une location
2. Rapport de rentabilite
3. Liste alphabetique
4. Rechercher
5. Modifier duree
6. Quitter
Votre choix: 3

--- LISTE PAR CLIENT ---
Alami_Sara - BMW_320 - 3360.00 MAD
Benali_Omar - Dacia_Logan - 1470.00 MAD
Chraibi_Omar - Clio_IV - 1134.00 MAD
```

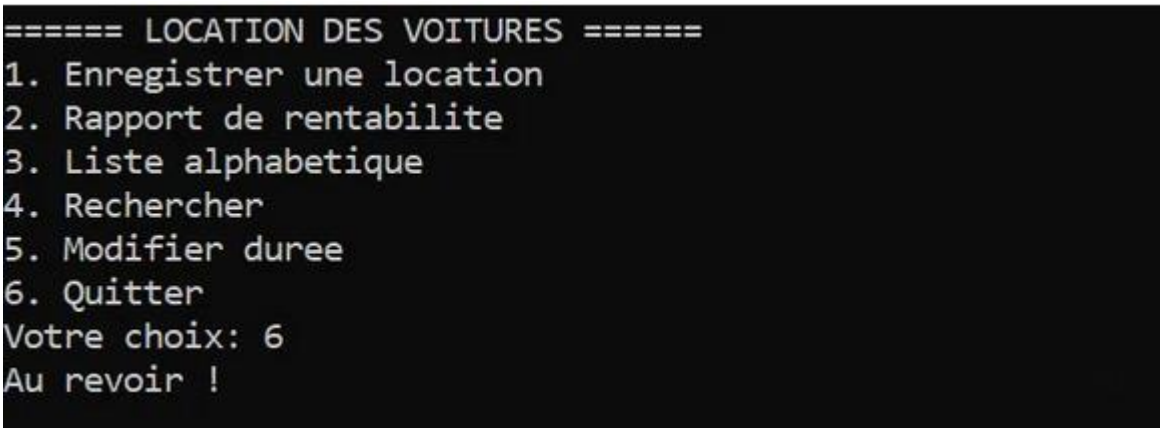
Choix4:

```
====== LOCATION DES VOITURES ======
1. Enregistrer une location
2. Rapport de rentabilite
3. Liste alphabetique
4. Rechercher
5. Modifier duree
6. Quitter
Votre choix: 4
Nom du client: Alami_Sara
Trouve: Alami_Sara - BMW_320 - 3360.00 MAD
```

Choix5:

```
====== LOCATION DES VOITURES ======
1. Enregistrer une location
2. Rapport de rentabilite
3. Liste alphabetique
4. Rechercher
5. Modifier duree
6. Quitter
Votre choix: 5
Immatriculation: 3000360
Nouvelle duree (jours): 30
Nouveau total mis a jour: 20160.00 MAD
```

Choix6:



9. Conclusion

En conclusion, ce programme de **gestion de location de voitures** repose sur **plusieurs fonctions** organisées qui permettent de **structurer** le **travail**, comme:

- **calculerTotal** pour déterminer le coût
- **definirTarif** pour attribuer le tarif et l'assurance selon la catégorie, ainsi que **enregistrer**, **rapport**, **trierParTotal**, **trierParNom**, **rechercher** et **modifier** pour gérer les différentes opérations.

Nous avons utilisé des conditions **if / else** pour **comparer** les **catégories** et **effectuer** des **choix**, ainsi que pour **vérifier** les **recherches** et les **modifications**.

De plus, des boucles **for** ont été utilisées pour **parcourir** le tableau, **trier** les données et **calculer** le chiffre d'affaires total.

Enfin, la structure **switch** dans le menu principal permet de **gérer** les différentes actions de l'utilisateur.

L'ensemble rend le programme **clair**, **organisé** et **efficace** pour gérer les locations et analyser la rentabilité.